Practice 09 — Load Balancer and a Self-Healing Fleet

Objective: put a load balancer in front of a group of identical web servers, watch it spread traffic across them, kill one and see the service keep running while the fleet **heals itself**. These are the skills from today's lecture.

Timebox: in class: ~50–55 min on Track A, ~35 min on Track B. If you've been stuck on one step for more than 10 minutes, stop and ask. Don't push through alone.

> **How you're checked:** this practice is done in class and checked **orally**. There is no report. When you finish, keep your terminal (and console, on Track A) open. The instructor will ask you to show the items in the *Definition of done* and to explain what you did.

Pick your track

You don't need AWS access to pass. Pick **one** track:

	Track A — real AWS	**Track B — no AWS**
Needs	AWS Academy Learner Lab (or your own account)	Docker (or Podman) and a bash shell (Linux, macOS, or WSL / Git Bash on Windows)
Load balancer	Application Load Balancer (ALB)	HAProxy container
Servers	EC2 instances in an Auto Scaling group	`nginx` containers (`docker compose` replicas)
Slow step	ALB and instances becoming healthy: 3–5 min	none

The ideas are the same on both tracks:

AWS (Track A)	Track B equivalent
Application Load Balancer	`lb` container running HAProxy
Listener on port 80	`frontend http_in` → `bind :80` (published as `8080`)
Target group	`backend web_servers`
Health check `GET /` every 10 s	`option httpchk GET /` with `inter 2s fall 2 rise 2`
Launch template (AMI, type, user data, SG)	The `web` service definition in `compose.yaml`
Auto Scaling group, desired = 2	`deploy: replicas: 3`, changed with `--scale web=N`
ASG replaces an unhealthy instance	`restart: always` restarts a crashed container
ALB finds new instances through the target group	HAProxy finds new replicas through Docker DNS (`server-template` + `resolvers docker`)
alb-sg → web-sg (port 80 only from the ALB)	Only `lb` publishes a port; `web` containers publish nothing
Two AZs	No equivalent: one machine, one "AZ"

Definition of done

Be ready to **show** each of these:

□ The load balancer's address in a browser or `curl`, and **several requests answered by different servers** (Track A: different instance IDs and AZs. Track B: different container hostnames).
□ The health status of every target (Track A: `describe-target-health` shows `healthy`. Track B: the HAProxy stats page shows `UP`).
□ The rule that lets traffic in (Track A: web-sg inbound 80 with **source = alb-sg**. Track B: only `lb` publishes a port).
□ One server killed **while the site keeps answering 200**, and the fleet back to full size (Track A: the ASG launched a replacement. Track B: the crashed container was restarted).
□ Scaling: Track A: a target-tracking policy on the ASG. Track B: the fleet scaled to 5 and back to 2, and HAProxy using all of them.
□ Cleanup done and shown.

Track A — real AWS (AWS Academy Learner Lab)

> **Using AWS Academy Learner Lab**
>
> 1. Open the **AWS Academy Learner Lab** course → **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to **AWS** turns green (1–3 min), then click

AWS to open the console.
2. Use region `us-east-1` (N. Virginia). Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed; other regions fail.
3. CLI: use the terminal on the lab page (the AWS CLI is already set up), or copy the credentials from **AWS Details** → **AWS CLI** → **Show** into `~/.aws/credentials` on your laptop. They change every session.
4. Key pair: don't create one. Use the lab's `vockey` : **AWS Details** → **Download PEM** saves `labsuser.pem` (only needed for the stretch).
5. Load balancers and Auto Scaling groups are usually allowed in Learner Lab, and the lab limits how many instances can run at once. Per the Learner Lab Readme, keep the ASG small (max 4). If a command is refused with `AccessDenied` or `UnauthorizedOperation`, switch to Track B.
6. When you're done, clean up (A8) and click **End Lab**. A load balancer bills every hour it exists, even with no traffic.

We use the **default VPC** to save time. Its subnets are public, so the instances get public IPs. In production, the ALB sits in public subnets and the instances in **private** subnets (week 8's design); the security groups below are the same either way.

A1. Network and security groups

```
export AWS_REGION=us-east-1

VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)

# Two default subnets = two different AZs (one default subnet per AZ)
read S1 S2 <<< $(aws ec2 describe-subnets \
  --filters Name=vpc-id,Values=$VPC Name=default-for-az,Values=true \
  --query 'Subnets[0:2].SubnetId' --output text)
aws ec2 describe-subnets --subnet-ids $S1 $S2 \
  --query 'Subnets[].[SubnetId,AvailabilityZone]' --output table

SG_ALB=$(aws ec2 create-security-group --group-name lab09-alb-sg \
  --description "lab09 ALB" --vpc-id $VPC --query GroupId --output text)
SG_WEB=$(aws ec2 create-security-group --group-name lab09-web-sg \
  --description "lab09 web servers" --vpc-id $VPC --query GroupId --output text)

# The internet may reach the ALB on 80
aws ec2 authorize-security-group-ingress --group-id $SG_ALB \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

# The web servers accept 80 ONLY from the ALB's security group
aws ec2 authorize-security-group-ingress --group-id $SG_WEB \
  --protocol tcp --port 80 --source-group $SG_ALB
```

A2. Launch template: what every server looks like

Save as `user-data.sh`:

```
#!/bin/bash
dnf install -y nginx
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo "Hello from $IID in $AZ" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

```
UD=$(base64 < user-data.sh | tr -d '\n')     # works on Linux and macOS

cat > lt.json <<EOF
{
  "ImageId": "resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64",
  "InstanceType": "t3.micro",
  "KeyName": "vockey",
  "SecurityGroupIds": ["$SG_WEB"],
  "UserData": "$UD",
  "MetadataOptions": {"HttpTokens": "required"},
  "IamInstanceProfile": {"Name": "LabInstanceProfile"}
}
EOF

aws ec2 create-launch-template --launch-template-name lab09-lt \
  --launch-template-data file://lt.json
```

The instance profile line is optional: remove it if the lab refuses it.

A3. Target group (with a health check)

```
TG=$(aws elbv2 create-target-group --name lab09-tg \
  --protocol HTTP --port 80 --vpc-id $VPC --target-type instance \
  --health-check-path / --health-check-interval-seconds 10 \
  --healthy-threshold-count 2 --unhealthy-threshold-count 2 \
  --query 'TargetGroups[0].TargetGroupArn' --output text)
```

A4. The load balancer and its listener

```
ALB=$(aws elbv2 create-load-balancer --name lab09-alb --type application \
  --scheme internet-facing --subnets $S1 $S2 --security-groups $SG_ALB \
  --query 'LoadBalancers[0].LoadBalancerArn' --output text)

aws elbv2 create-listener --load-balancer-arn $ALB --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn=$TG

DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB \
  --query 'LoadBalancers[0].DNSName' --output text)
echo $DNS
```

A5. The Auto Scaling group

```
aws autoscaling create-auto-scaling-group --auto-scaling-group-name lab09-asg \
  --launch-template 'LaunchTemplateName=lab09-lt,Version=$Latest' \
  --min-size 2 --desired-capacity 2 --max-size 4 \
  --vpc-zone-identifier "$S1,$S2" \
  --target-group-arns $TG \
  --health-check-type ELB --health-check-grace-period 120
```

Wait 3–5 minutes (the ALB becomes `active`, the instances boot and run user data), then:

```
aws elbv2 describe-target-health --target-group-arn $TG \
  --query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State]' --output table

for i in $(seq 1 8); do curl -s http://$DNS/; done
```

You should see **two different instance IDs in two different AZs**, taking turns.

Also try `curl http://<public-ip-of-one-instance>/`. It **times out**: web-sg only accepts port 80 from the ALB.

A6. Kill a server: the site stays up, the fleet heals

```
VICTIM=$(aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names lab09-asg \
  --query 'AutoScalingGroups[0].Instances[0].InstanceId' --output text)
aws ec2 terminate-instances --instance-ids $VICTIM

# Keep hitting the ALB: every answer should still be 200, from the survivor
for i in $(seq 1 10); do curl -s -o /dev/null -w '%{http_code} ' http://$DNS/; sleep 1; done; echo

# Watch the ASG notice and launch a replacement (takes a few minutes)
aws autoscaling describe-scaling-activities --auto-scaling-group-name lab09-asg \
  --max-items 3 --query 'Activities[].[StatusCode,Description]' --output table
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names lab09-asg \
  --query 'AutoScalingGroups[0].Instances[].
[InstanceId,AvailabilityZone,LifecycleState,HealthStatus]' \
  --output table
```

A7. A scaling policy

```
aws autoscaling put-scaling-policy --auto-scaling-group-name lab09-asg \
  --policy-name cpu-50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration \
  '{"PredefinedMetricSpecification":
{"PredefinedMetricType":"ASGAverageCPUUtilization"},"TargetValue":50.0}'

aws autoscaling describe-policies --auto-scaling-group-name lab09-asg \
  --query 'ScalingPolicies[].[PolicyName,PolicyType]' --output table
```

Target tracking creates the CloudWatch alarms for you. It adds instances when average CPU stays above 50% and removes them when it drops, always within min 2 and max 4.

Stretch (optional, if you have 10+ minutes): make it scale out. Allow SSH to web-sg from your IP, connect to one instance and burn CPU:

```
MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id $SG_WEB --protocol tcp --port 22 --cidr
$MYIP/32
ssh -i labsuser.pem ec2-user@<public-ip-of-one-instance>
# on the instance (t3.micro has 2 vCPUs):
yes > /dev/null & yes > /dev/null &
```

The alarm needs a few minutes of high CPU before it fires. Watch `describe-scaling-activities` for "Launching a new EC2 instance". Kill the load with `pkill yes` when you're done.

A8. Cleanup (required: the ALB bills per hour)

Order matters: the ASG first (it would keep replacing instances), then the ALB, then what they depended on.

```
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name lab09-asg --force-delete
# wait until the group is gone and its instances are terminated (1–3 min):
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names lab09-asg \
  --query 'AutoScalingGroups[].Status' --output text      # empty = gone

aws elbv2 delete-load-balancer --load-balancer-arn $ALB    # also deletes its listener
aws elbv2 wait load-balancers-deleted --load-balancer-arns $ALB
aws elbv2 delete-target-group --target-group-arn $TG
aws ec2 delete-launch-template --launch-template-name lab09-lt

# web-sg references alb-sg, so delete it first. If you get
# DependencyViolation, the ALB's network interfaces are still going away:
# wait a minute and retry.
aws ec2 delete-security-group --group-id $SG_WEB
aws ec2 delete-security-group --group-id $SG_ALB
```

Then click **End Lab**.

Track B — no AWS (Docker)

Every command works with `podman compose` instead of `docker compose`. Create the folder with exactly this name, so your containers are called `lab09-web-1`, `lab09-web-2`, …:

```
mkdir lab09 && cd lab09
```

B1. The web servers (the "launch template")

Save as `web.conf`. Each server answers with its own hostname, the way the EC2 page showed its instance ID:

```
server {
  listen 80;
  default_type text/plain;
  location / { return 200 "Hello from $hostname\n"; }
}
```

B2. The load balancer (HAProxy)

Save as `haproxy.cfg`:

```
resolvers docker
  nameserver dns1 127.0.0.11:53
  resolve_retries 3
  timeout resolve 1s
  timeout retry   1s
  hold valid      5s

defaults
  mode http
  timeout connect 3s
  timeout client  10s
  timeout server  10s

frontend http_in
  bind :80
  default_backend web_servers

backend web_servers
  balance roundrobin
  option httpchk GET /
  default-server inter 2s fall 2 rise 2
  server-template web 10 web:80 check resolvers docker init-addr none

frontend stats
  bind :8404
  stats enable
  stats uri /
  stats refresh 5s
```

- `backend web_servers` is the **target group**; `option httpchk GET /` is its **health check**: 2 failures mark a server down, 2 successes bring it back.
- `server-template web 10 web:80 ... resolvers docker` keeps up to 10 slots and fills them from Docker's DNS for the name `web`. When you scale the `web` service, HAProxy picks up the new replicas by itself, the same way an ALB picks up new instances in its target group.

B3. Wire it together

Save as `compose.yaml`:

```
services:
  web:
    image: nginx:alpine
    volumes:
      - ./web.conf:/etc/nginx/conf.d/default.conf:ro
    restart: always
    deploy:
      replicas: 3

  lb:
    image: haproxy:3.0-alpine
    volumes:
      - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    ports:
      - "8080:80"
      - "8404:8404"
    depends_on:
      - web
```

Only `lb` publishes ports. The `web` containers publish nothing, the same as web-sg only accepting traffic from alb-sg.

```
docker compose up -d
sleep 5
for i in $(seq 1 6); do curl -s localhost:8080; done
```

Three different hostnames take turns: **round robin**. Open http://localhost:8404 to see the stats page: every `webN` slot that has a container is `UP`, and the empty slots show `MAINT (resolution)`.

B4. Kill a server: the site stays up

```
docker stop lab09-web-2
sleep 5
for i in $(seq 1 6); do curl -s -o /dev/null -w '%{http_code} ' localhost:8080; done; echo
for i in $(seq 1 4); do curl -s localhost:8080; done
```

Every answer is still `200`, now from the two remaining servers. On the stats page one slot is no longer `UP`: Docker's DNS stopped returning the stopped container, so HAProxy shows that slot as `MAINT (resolution)`. (A server that is still in DNS but fails its health check shows `DOWN` instead.)

Now a **crash** instead of a manual stop. `restart: always` plays the role of the ASG replacing an unhealthy instance:

```
docker compose up -d                      # bring web-2 back first
C=lab09-web-1
docker inspect -f 'restarts={{.RestartCount}}' $C
docker exec $C sh -c 'kill 1'             # the main process dies
sleep 5
docker inspect -f 'restarts={{.RestartCount}} status={{.State.Status}}' $C
for i in $(seq 1 6); do curl -s -o /dev/null -w '%{http_code} ' localhost:8080; done; echo
```

`RestartCount` goes from 0 to 1 and the container is `running` again. A `docker stop` doesn't trigger a restart: Docker treats it as "you meant to stop it", just as setting an ASG's desired capacity lower isn't a failure.

B5. Scale out and in

```
docker compose up -d --scale web=5 --no-recreate
sleep 8
for i in $(seq 1 10); do curl -s localhost:8080; done | sort | uniq -c   # 5 hostnames

docker compose up -d --scale web=2 --no-recreate
sleep 8
for i in $(seq 1 6); do curl -s localhost:8080; done | sort | uniq -c    # 2 hostnames
```

You never touched `haproxy.cfg`: new replicas appeared in DNS and HAProxy added them, removed ones disappeared. In Track A, an Auto Scaling policy changes the number automatically from a metric; here you're the "policy".

B6. Cleanup

```
docker compose down
cd .. && rm -rf lab09
```

Be ready to explain

1. Why are the web servers' security group rules "port 80 from **alb-sg**" instead of "port 80 from 0.0.0.0/0"? (Track B: what plays that role?)
2. What does a health check do, and what happens to a target that fails it twice?
3. You terminated an instance. Who noticed, who stopped sending it traffic, and who replaced it? (ALB vs ASG — two different jobs.)
4. What do min, desired and max mean in an Auto Scaling group? What did the target-tracking policy change?
5. Why should the app be **stateless** behind a load balancer? What breaks if a user's session lives in one server's memory?

6. How does this setup survive the loss of a whole AZ? What would still be a single point of failure from week 8?

Common mistakes

- **Targets stay `unhealthy` (Track A).** The health check path returns something other than 200 (for example, user data hasn't finished installing nginx yet; wait 2–3 minutes), or web-sg doesn't allow port 80 from alb-sg.
- **`curl` to the ALB hangs (Track A).** The ALB is still `provisioning` (check `describe-load-balancers` → `State`), or alb-sg is missing the port 80 rule from `0.0.0.0/0`.
- **`503 Service Temporarily Unavailable` from the ALB (Track A).** The target group has no healthy targets yet. Wait for `describe-target-health` to show `healthy`.
- **Every response comes from the same instance (Track A).** Only one target is healthy, or the subnets you gave the ASG are in the same AZ.
- **The ASG keeps launching instances after you delete them (Track A).** You terminated instances instead of deleting the ASG. Delete the ASG first (`--force-delete`), then everything else.
- **`DependencyViolation` deleting a security group (Track A).** Delete web-sg before alb-sg, and wait until the ALB is fully deleted.
- **Containers are called something else, so `docker stop lab09-web-2` fails (Track B).** Compose names containers after the folder. Run it from a folder called `lab09`, or use `docker compose ps` to see the real names.
- **New replicas don't get traffic (Track B).** HAProxy re-reads Docker DNS every few seconds. Wait 5–10 seconds, and check that the `resolvers` section and `resolvers docker` on the `server-template` line are both there.